

Axios – AWS-Native Architecture & Workflow

Built for AWS x Bharat Builds – First Commit

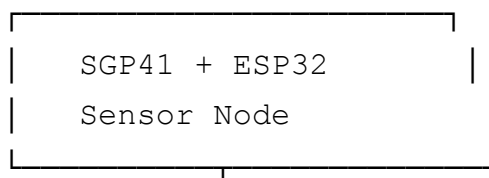
Your sketch is the right shape. This fleshes it out into a design that (a) actually maps every Phase 10–20 function from the roadmap onto specific AWS services, (b) fixes a couple of things in the original sketch that would bite you at demo time, and (c) gives you a strong "we used AWS properly, not just as a database" story for judges – which matters a lot in an AWS-sponsored hackathon.

1. What changes from your sketch, and why

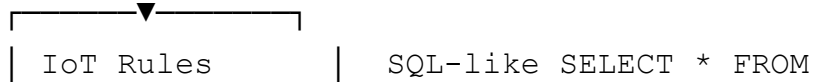
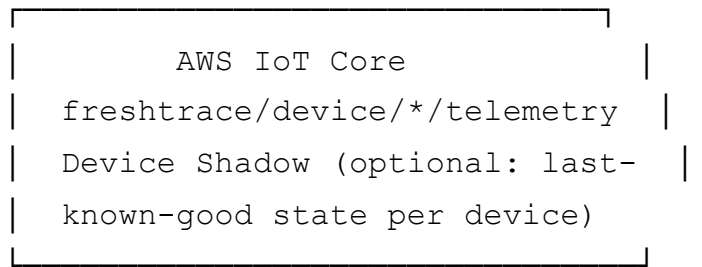
Your sketch	Revised	Why
Kinesis/SQS (one box)	Kinesis Data Streams (real-time path) + Kinesis Data Firehose (archive path)	These solve different problems. Streams gives your processor ordered, replayable, low-latency records. Firehose is purpose-built to batch-write to S3 cheaply without you writing that logic yourself. Using both is also two distinct, correctly-used AWS services instead of one overloaded box.
IoT Rule → S3 directly	IoT Rule → Firehose → S3	A direct S3 IoT action writes one object per message (or per small batch) – expensive and creates an unqueryable mess of tiny files. Firehose batches by time/size before writing, and gzips automatically.
ECS Fargate holds all processing logic	ECS Fargate (stream consumer) + DynamoDB (hot per-device state)	Fargate is right for a long-running stream consumer (this part of your sketch was already correct). But baseline/EMA state needs to survive task restarts and support future horizontal scaling – that state belongs in DynamoDB, not just in-process memory.

Your sketch	Revised	Why
RDS only	RDS (Aurora PostgreSQL) for relational data + Amazon Timestream for raw/processed time-series telemetry	Sensor readings are exactly what Timestream is built for (automatic time-based partitioning, cheap high-cardinality writes, fast range queries for the dashboard's timeline chart). Sessions, events, and state snapshots are relational and belong in Aurora. Splitting these is a meaningfully better fit, not just more services for the sake of it.
API → Dashboard (implied REST)	API Gateway + Lambda for REST, AppSync for live dashboard updates	A polling REST API works but a spoilage dashboard is fundamentally a live-updating thing. AppSync (GraphQL subscriptions) pushes state changes to the dashboard the moment Fargate writes them — much stronger demo moment than a 5-second poll loop.
(not present)	EventBridge as the central event bus	Your Phase 17 evidence engine already produces discrete named events (<code>SPOILAGE_ONSET_CONFIRMED</code> , etc.). Instead of writing those only to Aurora, publish them to EventBridge too — it's the natural AWS-native fan-out point for anything you want to bolt on later (SNS alert, Step Functions workflow, a Slack webhook for the demo) without touching the processor code again.
(not present)	CloudWatch (logs, metrics, alarms)	Judges notice when a team can show a metrics dashboard proving the pipeline is actually healthy, not just a nice-looking frontend.
(not present, optional)	Amazon Bedrock for narrative evidence summaries	Turns a raw event list into a plain-English "why is this shipment high-risk" paragraph on the report export (§9.6 of the roadmap). Optional, but it's a genuinely useful feature <i>and</i> adds a modern AWS AI service to your stack — worth it if time allows.

2. Full architecture diagram



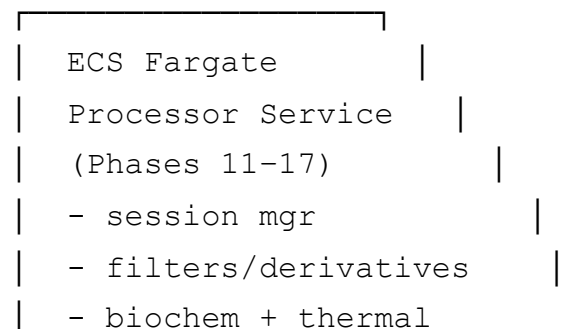
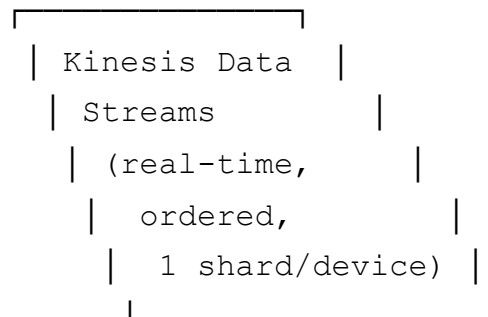
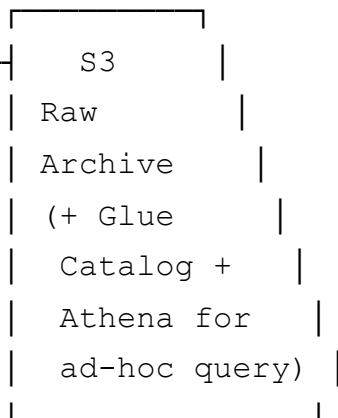
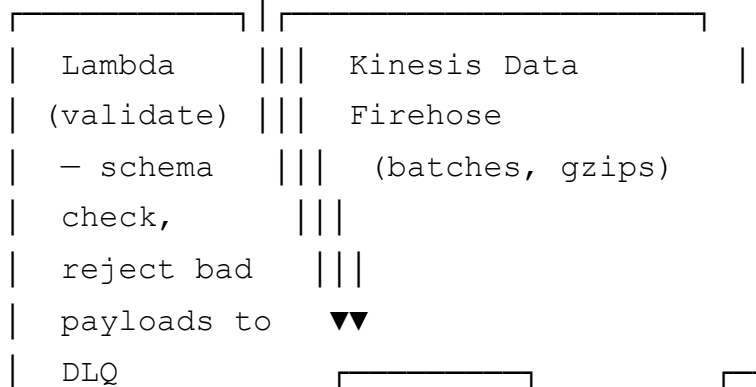
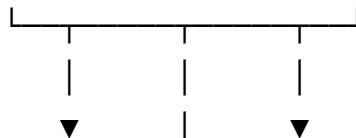
| MQTT (TLS, X.509 device cert)

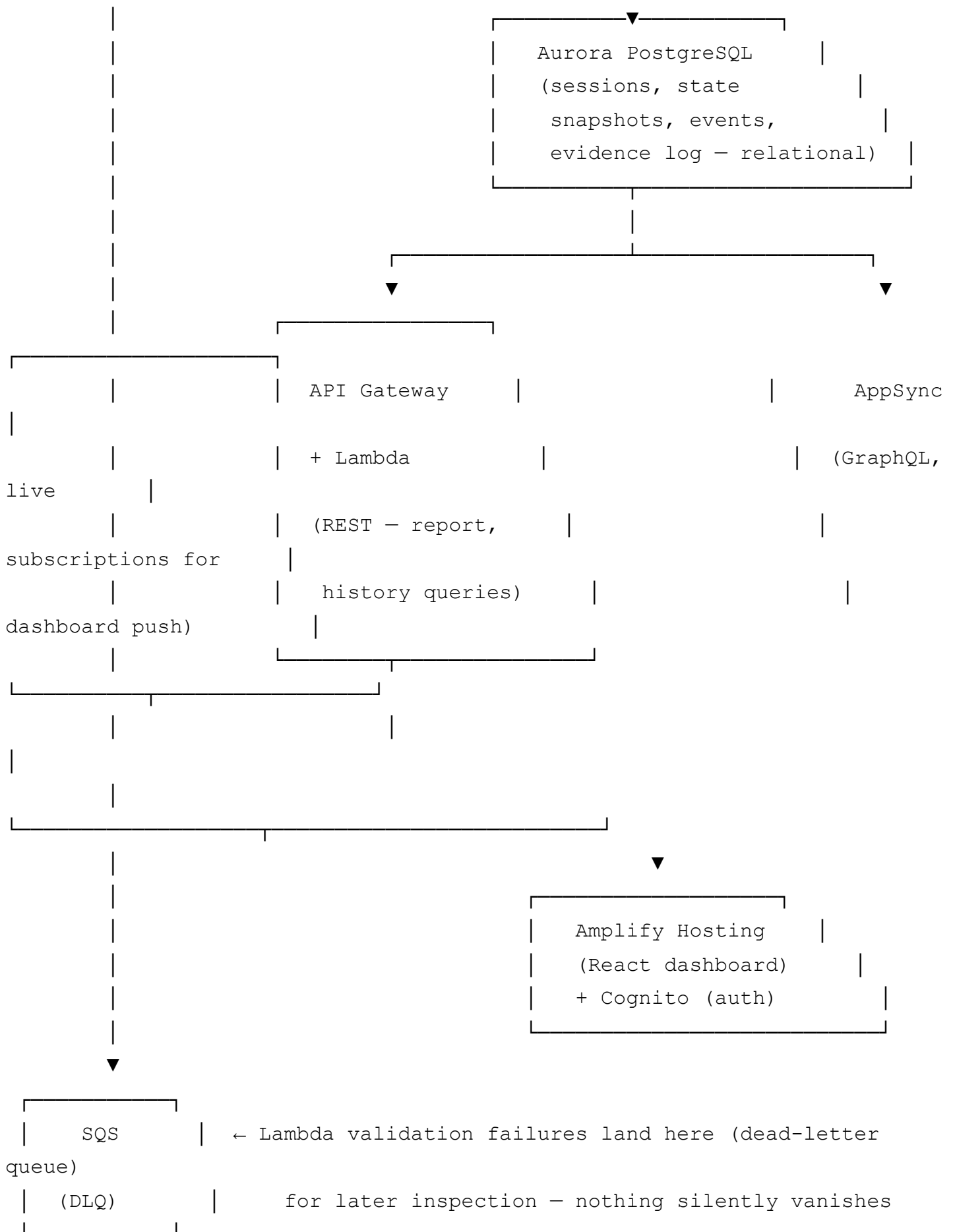


'freshtrace/device/*/telemetry'

| Engine | - fans out to 3 targets in parallel, no custom

code





(optional, if time allows)

Aurora events → Bedrock (Claude via Bedrock) → natural-language
evidence summary for report export

3. Workflow, step by step

1. **Device** → **IoT Core**. ESP32 publishes to `freshtrace/device/{device_id}/telemetry` over MQTT/TLS, authenticated with a per-device X.509 certificate provisioned through IoT Core's device registry. This is already how you're connected — nothing changes here.
2. **IoT Rule fan-out**. One SQL-style IoT Rule (`SELECT * FROM 'freshtrace/device/*/telemetry'`) triggers three actions on every message, with no code running in the rule itself:
 - Action A → **Lambda** (lightweight validation — schema/range checks from Phase 10 §5.2 of the roadmap). Valid messages are re-published to a `validated` topic; invalid ones go to an **SQS dead-letter queue** with the rejection reason attached, so nothing is silently dropped.
 - Action B → **Kinesis Data Firehose**, which buffers and batch-writes to **S3** as the permanent raw archive (partitioned by device/date). An AWS Glue Crawler run over this bucket registers a schema, so you can run ad-hoc Athena SQL over historical raw data — a nice thing to demo if a judge asks "can you show me the raw sensor data for this window."
 - Action C → **Kinesis Data Streams**, one stream (or one shard per device if you want to future-proof for a fleet), which is the low-latency path the processor actually consumes from.
3. **ECS Fargate Processor** runs as a long-lived service with a Kinesis Client Library consumer. This is the right shape for your sketch's original idea — Fargate rather than Lambda specifically because Phases 11–17 need continuous per-device state (running baseline, EMA, volatility window) that's awkward to carry across stateless Lambda invocations. It:
 - Maintains session lifecycle and baseline (Phase 11) — hot state (running mean/stddev, last EMA value, last N readings for outlier checks) lives in **DynamoDB**, keyed by `device_id`, so the processor can restart or scale to multiple tasks without losing state.
 - Writes filtered/processed readings to **Timestream** (this is your time-series query backend for the dashboard's charts).
 - Runs the biochemical engine, thermal engine, onset state machine, RUL estimator, and fusion logic (Phases 12–16) in-process, since they all operate on the same per-reading stream.
 - Writes session records, current state snapshots, and evidence events to **Aurora PostgreSQL** (relational, good for the joins the report export needs).
 - Publishes every named event (Phase 17) to **EventBridge**.
4. **EventBridge fan-out**. From one event bus:
 - A rule matching `SPOILAGE_ONSET_CONFIRMED` or `*_RISK_HIGH` → **SNS** → email/SMS or a webhook into a Slack channel for the demo ("look, it just paged us live").
 - A rule matching any event → also lands in CloudWatch Logs for observability.

- This is also where you'd hang a Step Functions workflow later if you wanted a multi-step response (e.g. auto-flag a shipment for inspection) — you don't need to build that for the hackathon, but having EventBridge in place means you can say "the architecture supports it" credibly.

5. API layer, two paths for two purposes:

- **API Gateway + Lambda** for anything request/response — historical queries, the report export endpoint (Phase 18's `/devices/{id}/report`), device list.
- **AppSync** for anything that should update live on screen — current risk state, onset alerts. AppSync subscriptions push straight to the dashboard the moment Aurora/Timestream is written, no polling loop, no manual WebSocket plumbing.

6. **Amplify Hosting + Cognito.** The React dashboard (Phase 19 of the roadmap) is hosted on Amplify, with Cognito handling auth if you want a login screen for the demo (or skip Cognito entirely if the hackathon demo is single-tenant and auth isn't the point — don't spend hackathon hours on login screens unless judges specifically care).

7. **(Optional, high-value if time allows) Bedrock.** On report export, send the session's event list + current state object to Claude via Bedrock with a prompt like "explain in plain English why this shipment is at X% risk, referencing these events" — turns your evidence log into a paragraph a non-technical buyer/inspector can read. This is a strong differentiator for a food-safety-facing product and it's a genuine, non-gimmicky use of an AI service, which tends to score better with judges than a bolted-on chatbot.

4. Why this mapping (for your pitch / judging Q&A)

- **Every AWS service here is doing the job it's actually built for**, not used as a generic substitute for something simpler — that's the difference between "we used a lot of AWS" and "we used AWS well." Be ready to explain *why* Timestream vs. Aurora, and why Fargate vs. Lambda for the processor, in exactly those terms (state, latency, query shape) — judges in an AWS-branded hackathon usually probe this specifically.
- **IoT Core → Rules Engine → parallel fan-out** is the idiomatic AWS IoT pattern (this part of your original sketch was already correct) and costs you nothing extra to implement since it's declarative, not code.
- **DynamoDB for hot state / Timestream for time-series / Aurora for relational** is polyglot persistence used correctly — each store holds the data shape it's best at, which is a legitimate architectural story, not overengineering, given the actual access patterns (per-device point lookups, time-range scans, and relational joins for reporting are three different query shapes).
- **EventBridge as the seam** means your evidence system (which the roadmap already treats as a first-class differentiator) is wired into the rest of AWS's ecosystem instead of being a dead-end

table.

5. Security & IAM (worth having answers ready for, even if not fully implemented by demo time)

- Each IoT device uses a unique X.509 certificate with an IoT policy scoped to publish only on its own topic (`freshtrace/device/${iot:Connection.Thing.ThingName}/telemetry`) — prevents one compromised device from publishing as another.
 - Fargate task role: least-privilege — read from its Kinesis stream, read/write its DynamoDB table, write to Timestream and Aurora, publish to EventBridge. No S3 access needed (that's Firehose's job, not the processor's).
 - Lambda validation function role: publish to the `validated` topic and SQS DLQ only.
 - Aurora and DynamoDB in private subnets; only the API Gateway/Lambda and AppSync resolvers (via VPC access if needed) can reach them — the dashboard never talks to the database directly.
-

6. Build order (mirrors the roadmap's dependency-first sequencing, now AWS-specific)

1. IoT Rule + Lambda validation + DLQ (you already have the IoT Core connection — this is the hardening layer).
2. Firehose → S3 archive path (cheap to stand up, unblocks nothing downstream but gets your audit trail running from day one).
3. Kinesis Data Streams + Fargate consumer skeleton (just prove messages flow end-to-end before adding any scoring logic).
4. DynamoDB hot-state schema + baseline/filter logic inside Fargate.
5. Timestream writes for processed readings.
6. Biochemical/thermal/onset/RUL/fusion logic inside the same Fargate service (this is where the roadmap's Phases 12–16 math actually lives).
7. Aurora schema for sessions/events/state snapshots + EventBridge publishing.
8. API Gateway + Lambda for REST endpoints.
9. AppSync subscriptions wired to Aurora/Timestream writes.
10. Amplify dashboard, built against real data from step 8–9 onward.
11. SNS alert rule + CloudWatch dashboard (fast to add, strong demo payoff).
12. Bedrock narrative summary, if time remains.